Caching

什麼是 Caching？

在系統設計面試中，幾乎每次討論到高讀取流量，快取都會出現。你的資料庫成了瓶頸，延遲開始爬升，面試官在等你說出那個關鍵詞：cache。

從 Postgres 讀一個用戶個人資料可能需要 50 毫秒，但從 Redis 這樣的記憶體快取讀取只需要 1 毫秒，足足快了五十倍。資料庫把資料存在磁碟上，每次查詢都要付出磁碟存取的代價；記憶體離 CPU 近得多，直接繞過了這個瓶頸。

快取對可擴展的系統至關重要：它降低資料庫的負載，並且大幅削減延遲。但它也帶來了快取失效和故障處理的新挑戰。

這篇講義涵蓋快取的基本概念、什麼時候用、在哪裡用、常見的陷阱，以及如何在面試中清楚地說明你的快取策略。

在哪裡快取

多數工程師聽到快取，腦子裡馬上浮現的是放在 application 和資料庫之間的 Redis 或 Memcached。這確實是最常見的快取形式，也是面試官最在意的那一種。

但快取其實存在於系統的多個層次：瀏覽器在快取、CDN 在快取、應用程式在快取，就連資料庫本身也有內建的快取層。

讓我們看看幾個主要的快取位置、各自存在的理由，以及什麼時候適合使用。

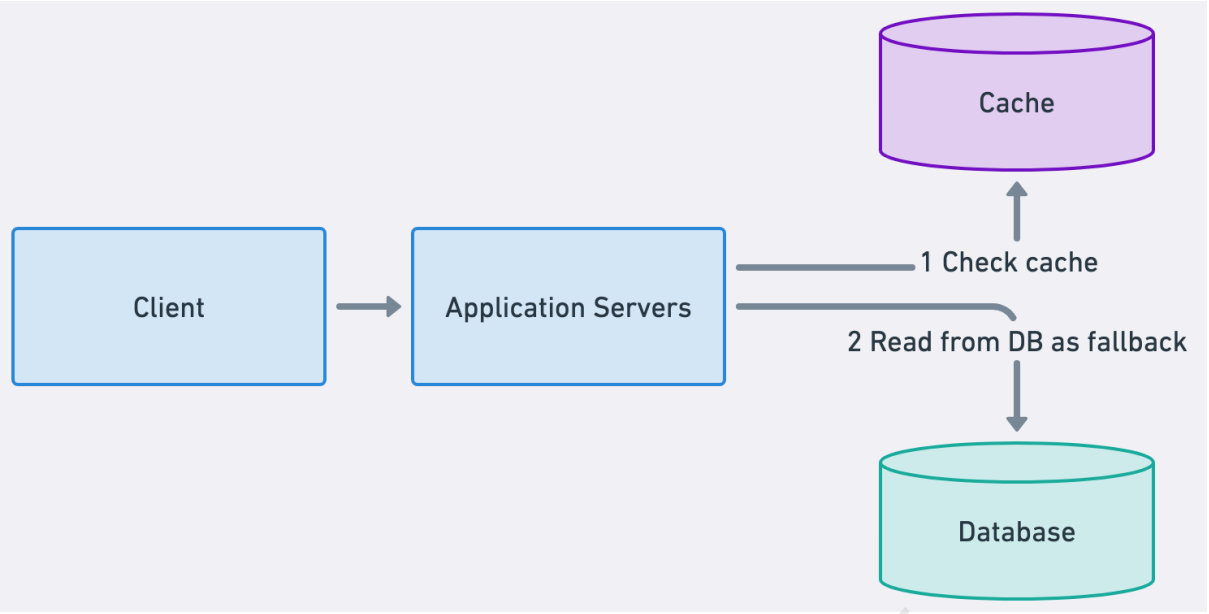
外部快取（External Caching）

外部快取是一個獨立的快取服務，你的 application 透過網路與它溝通。這是大多數人說到快取時心裡想的那種東西，把頻繁存取的資料存在 Redis 或 Memcached 裡，這樣就不需要每次都打到資料庫。

外部快取的擴展性很好，因為所有 application server 都可以共用同一個快取。它也支援 LRU 這樣的淘汰策略，以及透過設定過期時間（TTL，Time to Live），讓記憶體用量維持在可控的範圍內。

在系統設計面試中，討論快取策略時，用 Redis 做外部快取是預設答案。因為它在效能、共享性、操作成熟度之間取得了最好的平衡。面試官對任何高流量系統都預期你提到它。先從這裡出發，再根據題目需要加上 CDN 或客戶端快取這類其他層次。

buildmoat.org



CDN（Content Delivery Network）

CDN 是一個地理上分散的伺服器網路，把內容快取在靠近用戶的地方。使用者的請求不需要每次都跑到你的源站（origin server），CDN 在全球各地的邊緣伺服器（edge server）上存放你內容的副本。

Cloudflare、Fastly、Akamai 這類現代 CDN 能快取的東西遠不只靜態檔案，它們也能快取公開的 API 回應、HTML 頁面，甚至在邊緣執行邏輯來個人化內容或在請求抵達你的伺服器之前做安全檢查。但 CDN 最常見、效益最顯著的用途，還是媒體資源的傳遞。

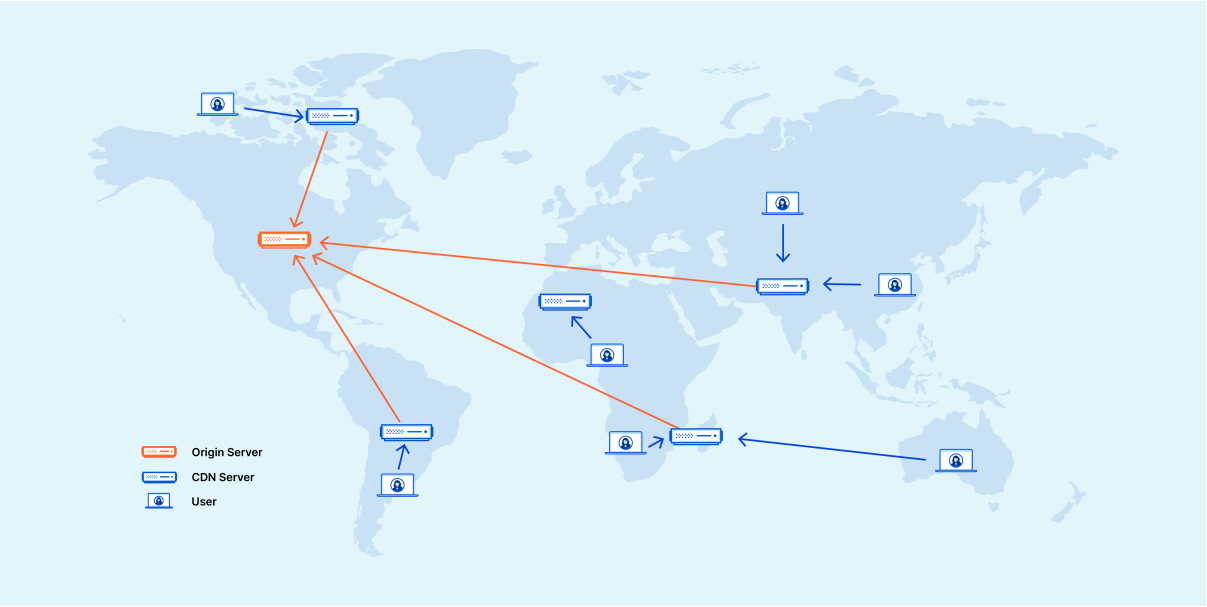
運作方式：

1. 用戶向你的 App 請求一張圖片
2. 請求被路由到最近的 CDN 邊緣伺服器
3. 如果那裡有快取，直接回傳
4. 如果沒有，CDN 從你的源站取得，存起來，再回傳
5. 同一區域的未來用戶直接從 CDN 拿到圖片，幾乎瞬間完成

沒有 CDN，每個圖片請求都要跑到你的後端伺服器。如果你的伺服器在維吉尼亞州，用戶在印度，光延遲就增加了 250 到 300 毫秒。有了 CDN，同一張圖片從附近的邊緣伺服器傳遞，只需要 20 到 40 毫秒，這個差距非常大。

雖然現代 CDN 可以快取 API 回應和動態內容，但在系統設計面試中，引入 CDN 最穩妥的理由是你的系統需要大規模傳遞靜態媒體資源（e.g. .png, .jpg, .svg）。先從這個理由開始，再視題目需求延伸。

buildneat.org

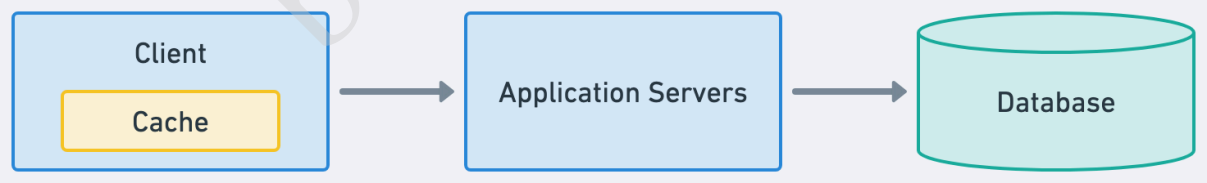


客戶端快取（Client-Side Caching）

客戶端快取把資料存在靠近請求方的地方，避免不必要的網路呼叫。通常指的是用戶的裝置，例如瀏覽器（HTTP cache、localStorage）或使用本地記憶體及裝置儲存空間的行動 App。

但它也可以指 client library 內部的快取。例如 Redis 的 client 會快取 cluster 的 metadata，記錄哪些節點在 cluster 中、哪些 slot 分配給哪些節點。這樣 client 就能直接把請求路由到正確的節點，而不需要每次操作都向 cluster 查詢。

對於面向用戶的快取，你從後端能控制的程度有限，資料可能過時，失效也更難處理。Strava App 在你沒有網路時把跑步資料留在裝置上，等之後再同步；瀏覽器重複使用之前從磁碟下載的圖片，也是一種快取。



行程內快取（In-Process Caching）

大多數面試者，甚至很多工程師，忽略了一件事：伺服器跑在有大量記憶體的機器上，而且隨著硬體進步，這個趨勢越來越明顯。你可以直接在 application 行程內快取資料，而不是每次都呼叫 Redis 或資料庫。

概念很簡單：如果你的服務一遍又一遍地請求同樣的小塊資料，就把它們存在行程內的本地快取裡。從本地記憶體讀取甚至比從 Redis 讀取還快，因為完全不需要任何網路呼叫。

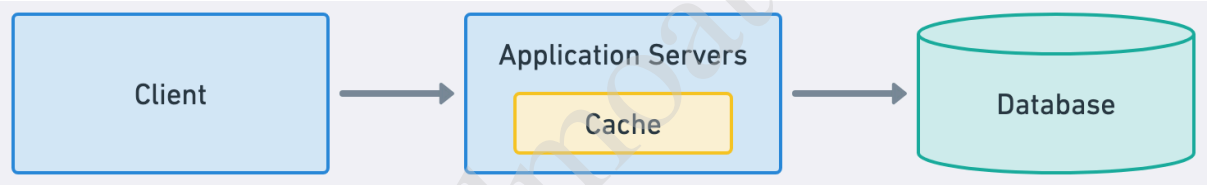
buildpat.org

這種輕量級快取適合頻繁被請求的小塊資料，例如：

- 設定值
- Feature flag
- 小型參考資料集
- 熱門 key
- Rate limiting 計數器
- 預先計算的結果

行程內快取速度極快，但有明顯的限制：每個 application instance 有自己的快取，快取的資料不在各台伺服器之間共享。如果一個 instance 更新或使某個快取值失效，其他 instance 不會知道。

把行程內快取用在很少改變、被頻繁存取的小塊資料上。它對速度很有幫助，但不是 Redis 的替代品。在系統設計面試中，只在你已經介紹了外部快取之後，才提這個作為優化層。



buildmoat.org

快取架構模式

並不是所有快取的運作方式都一樣。你讀寫快取的方式影響效能、一致性和複雜度。以下是你在系統設計面試中應該掌握的四種核心快取模式。

Cache-Aside（旁載快取）

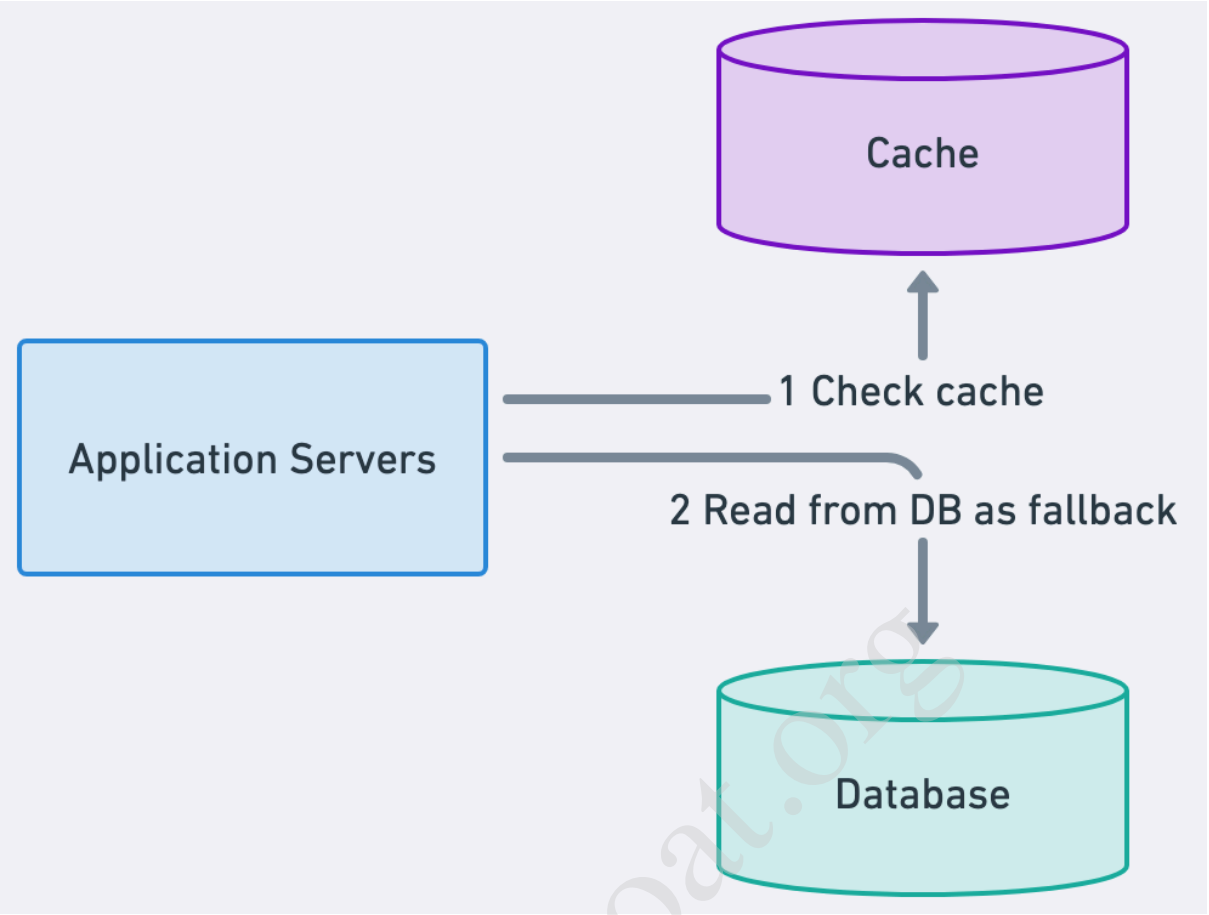
這是最常見的快取模式，也是你在面試中應該預設使用的那個。

運作方式：

1. Application 先查快取
2. 如果資料在，直接回傳
3. 如果不在，從資料庫取，存入快取，再回傳

Cache-aside 只在需要時才快取資料，讓快取保持精簡。缺點是 cache miss 會造成額外的延遲。

如果你只記得一種快取模式，記 cache-aside。



Write-Through（同步寫穿）

使用 write-through 時，application 只寫入快取，快取再同步地把資料寫到資料庫，整個寫入操作要等到快取和資料庫都更新完才算完成。

實際上這需要支援 write-through 的快取實作，例如有 data store 插件的快取 library。Redis 本身原生不支援 write-through，需要 application 程式碼或框架來實作這個模式。

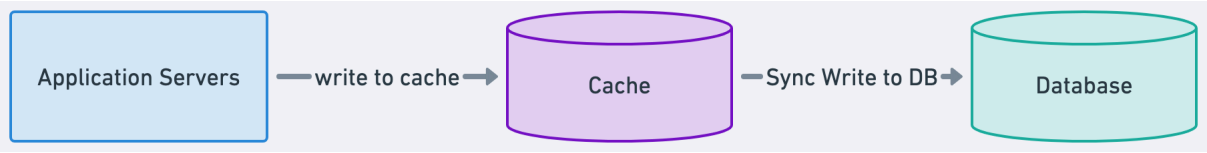
取捨在於寫入更慢，application 必須等待快取更新和資料庫寫入都完成。Write-through 也可能用那些再也不會被讀取的資料污染快取。

Write-through 仍然有雙重寫入（dual-write）的問題：如果快取更新成功但資料庫寫入失敗，或反過來，兩個系統就可能進入不一致的狀態。你需要重試邏輯、錯誤處理，或者接受在沒有分散式 transaction 的情況下，完美的一致性很難達到。

在系統設計面試中，write-through 比 cache-aside 少見，因為它需要特殊的快取基礎設施，而且仍然有一致性的邊緣情況。

適用場景：讀取必須永遠回傳新資料，而且系統可以接受稍微慢一點的寫入。

buildmoat.org

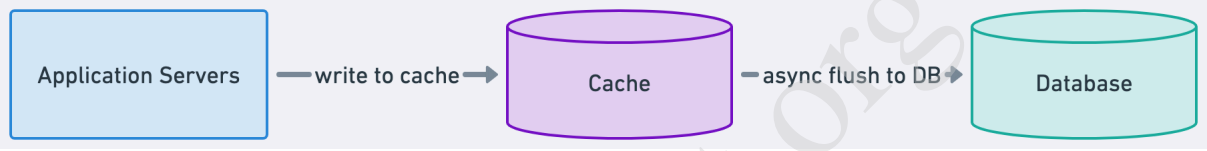


Write-Behind（非同步回寫）

使用 write-behind 時，application 只寫入快取，快取在背景非同步地把資料批次寫到資料庫。

這讓寫入非常快，但引入了風險：如果快取在把資料 flush 到資料庫之前崩潰，資料就遺失了。

適用場景：需要高寫入吞吐量，且可以接受最終一致性的工作負載。常見於分析和 metrics pipeline。



buildroat.org

Read-Through（讀穿快取）

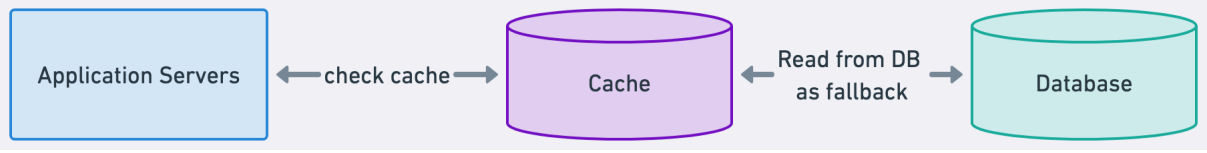
使用 read-through 時，快取充當智慧代理層，application 不直接和資料庫溝通。當 cache miss 發生，是由快取本身去資料庫取資料、存起來，再回傳給 application。

這是 write-through 的讀取端對應版本。在這兩種模式中，快取都充當處理資料庫操作的中間層，read-through 管理讀取，write-through 管理寫入。系統常常把兩者結合使用。

這個模式把快取邏輯集中起來，但增加了複雜度，通常需要特殊的快取 library 或服務，在實際應用中比 cache-aside 少見。

CDN 本質上是一種 read-through 快取——當 CDN 發生 cache miss，它從你的源站取得資料、快取結果，再回傳。但對於使用 Redis 的 application 層快取，cache-aside 遠比 read-through 普遍。

一般而言，除非你在討論 CDN 或類似的基礎設施，否則在系統設計面試中很少有理由主動提出這個模式。



快取淘汰策略

快取的記憶體有限，所以當快取滿了，需要有策略來決定該移除哪些項目。這些策略叫做淘汰策略（eviction policy）。

LRU（最近最少使用，Least Recently Used）：移除最久沒有被存取的項目。用 linked list 或 ring buffer 追蹤存取順序，可以在常數時間內移除最舊的項目。因為它對大多數「最近用過的資料很可能再次被用」的工作負載都能良好適應，是許多系統的預設選擇。

LFU（最不常使用，Least Frequently Used）：移除被存取次數最少的項目。為每個 key 維護一個計數器，移除頻率最低的那個。部分實作使用近似 LFU 來避免精確頻率追蹤的成本。適合某些 key 長期持續熱門的場景，例如熱門影片或排行榜播放清單。

FIFO（先進先出，First In First Out）：只根據插入時間，移除最早進入快取的項目。可以用簡單的 queue 實作，但它忽略了使用模式，可能移除還在被熱用的項目，所以除了簡單的快取層，在真實系統中很少使用。

TTL（存活時間，Time To Live）：TTL 本身不是淘汰策略，而是為每個 key 設定過期時間，移除太舊的項目，常和 LRU 或 LFU 搭配使用，在資料新鮮度和記憶體用量之間取得平衡。只要資料必須最終刷新（例如 API 回應或 session token），TTL 就是必備的。

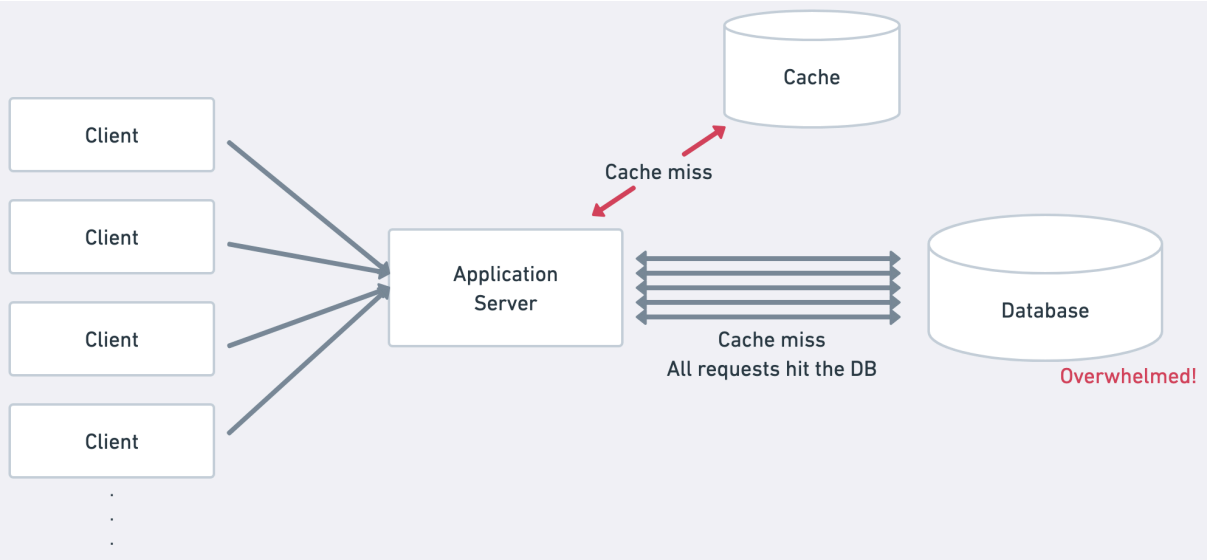
常見的快取問題

快取讓系統更快，但也引入了新的故障模式。這些問題在真實的大規模系統中都會出現，面試官也常用它們來測試你是否真的理解快取的取捨，而不只是知道它的好處。如果你在面試中提到快取，也應該展示你能應對這些邊緣情況。

快取雪崩（Cache Stampede / Thundering Herd）

當一個熱門的快取項目過期，大量請求同時嘗試重建它時，就發生了快取雪崩。有一段短暫的窗口——就算只有一秒——所有請求都 miss 快取，直接打到資料庫。原本一個查詢，瞬間變成幾百甚至幾千個，可能把資料庫打垮。

louiscnoat.org



舉個例子：你的系統快取首頁 feed，TTL 是 60 秒。快取在 12:01:00 整點到期，那個瞬間的所有請求全部 miss，同時打到資料庫。如果流量夠高，這個突刺可以讓資料庫過載，引發連鎖故障。

應對方式：

請求合併（Request coalescing / Single flight）：只讓一個請求去重建快取，其他請求等待那個結果。這是最有效的解法。

快取預熱（Cache warming）：在熱門 key 過期之前主動刷新。這只在用 TTL 過期的場景下有效；如果你是在寫入時失效快取，預熱無法防止雪崩。

快取一致性（Cache Consistency）

快取一致性問題是系統設計面試中最常被討論的問題之一。它發生在快取和資料庫對同一份資料回傳不同值的時候。這很常見，因為大多數系統從快取讀取，但先寫到資料庫——這製造了一個快取還拿著舊資料的窗口。

舉例來說，如果用戶更新了他們的大頭照，新的值被寫入資料庫，但舊的值可能還在快取裡。其他用戶在快取刷新之前可能還是看到舊的大頭照。

沒有完美的解法——你要根據資料必須有多新鮮來選擇策略：

寫入時失效快取：更新資料庫後刪除快取項目，讓下次讀取時以新資料填充快取。

短 TTL 容許過時資料：如果可以接受最終一致性，就讓稍微過時的資料暫時存在。

接受最終一致性：對 feed（動態牆、通知列表）、metrics（統計數句）、分析資料而言，短暫的延遲通常沒問題。

熱 Key（Hot Keys）

buildnoat.org

熱 key 是一個比其他所有項目收到多得多流量的快取項目。就算整體 cache hit rate 很高，單一熱 key 也可能讓某個快取節點或某個 Redis shard 過載，變成瓶頸。

舉例來說：如果你在做 Twitter，每個人都在看 Taylor Swift 的個人資料，她的 user data 快取 key（`user:taylorswift`）可能每秒收到幾百萬個請求。就算其他一切都運作正常，這單一個 key 就能把單台 Redis 節點打掛。

應對方式：

複製熱 key：把同樣的值存在多個快取節點上，把讀取負載分散出去。

加行程內備援快取：把極端熱門的值存在行程內，避免一直打 Redis。

套用 Rate Limiting：對特定 key 的異常流量模式踩煞車。

熱點快取（Hotspot Cache）

即便快取系統本身非常快，某些極度熱門的資料仍可能吸引大量請求，集中壓力到單一節點。為了避免單點瓶頸，可以將這些熱 key 複製到多個快取節點，分散流量負載。同時，不要讓各個副本的過期時間完全相同，以免它們同時過期，造成快取雪崩（Cache Stampede）。

系統設計面試中怎麼談快取

快取幾乎在每場系統設計面試中都會出現，所以知道什麼時候提出來、怎麼有條理地說明，非常重要。

什麼時候提到快取

不要一上來就直接說快取——你需要先確立為什麼有必要用它。

在你發現以下這些問題時提出快取：

讀取密集的工作負載：「我們要服務一千萬日活用戶，每人每天發出二十個請求，也就是每天兩億次讀取打到資料庫。就算有 index，每個查詢還是要 20 到 50 毫秒。快取可以把這降到 2 毫秒以下，同時把大部分負載從資料庫移開。」

昂貴的查詢：「計算用戶的個人化 feed 需要 join 貼文、追蹤關係和按讚記錄跨多張表，這個查詢要跑 200 毫秒。我們可以把算好的 feed 快取 60 秒，從 Redis 以 1 毫秒回傳。」

資料庫 CPU 過高：「我們的資料庫在尖峰時段 CPU 就達到 80%，幾乎都是在服務讀取，而且同樣的查詢一遍一遍地跑。把熱門查詢快取起來，可以把資料庫負載降低 70 到 80%。」

延遲需求：「我們需要 API 回應在 10 毫秒以內。資料庫查詢現在要 30 到 50 毫秒。必須快取。」

buildmoat.org

模式很簡單：**確認效能問題、用大概的數字量化它、說明快取如何解決它。**

如何介紹你的快取策略

一旦確立了需要快取，按照以下步驟有條理地說明你的快取策略：

第一步：確認瓶頸

先指出快取要解決的具體問題。是資料庫負載？查詢延遲？昂貴的計算？說清楚什麼東西慢、為什麼慢。

「用戶個人資料查詢在尖峰時段每秒打到資料庫五百次，每個查詢要 30 毫秒，這是我們的瓶頸。」

第二步：決定快取什麼

不是所有東西都應該快取。專注在讀取頻繁、不常變動、而且取得或計算成本昂貴的資料上。

「我們要快取用戶個人資料，因為每次頁面載入都會讀取，但只有用戶修改設定時才會更新。我們也要快取熱門貼文 feed，因為它需要昂貴的聚合計算，但每分鐘刷新一次就夠了。」

想清楚快取 key 的設計。`user:123:profile` 對應用戶個人資料；`trending:posts:global` 對應全站熱門貼文。

第三步：選擇快取架構

選一個符合你的一致性需求的快取模式：

「我用 cache-aside。讀取時先查 Redis，有就回傳；沒有就查資料庫，把結果存進 Redis，再回傳。」

如果你在處理圖片或影片這類靜態內容，提 CDN 快取。如果有被猛打的極端熱 key，提行程內快取作為優化層。

第四步：設定淘汰策略

說明你怎麼管理快取大小。LRU 是穩妥的預設答案，TTL 是防止資料過時的必備手段。

「我們用 Redis 的 LRU 淘汰，並且對用戶個人資料設定 10 分鐘的 TTL。這讓快取不會無限成長，同時確保個人資料不會過時太久。如果用戶更新了個人資料，我們立刻讓那個快取項目失效。」

第五步：說明缺點

快取帶來了複雜度，展示你想過取捨。

快取失效：你如何讓快取資料保持新鮮？寫入時失效、靠 TTL，還是接受最終一致性？

buildmoat.org

「用戶更新個人資料時，我們刪除對應的快取項目，讓下次讀取從資料庫取得新資料。」

快取故障：如果 Redis 掛了怎麼辦？資料庫會被突然暴增的流量打垮嗎？

「如果 Redis 不可用，請求會降級到直接查資料庫。我們會加 circuit breaker，避免雪崩式的請求把資料庫壓垮。也可以考慮保留一個小的行程內快取作為最後一道防線。」

快取雪崩：某個熱門快取項目過期，一千個請求同時嘗試重抓時怎麼辦？

「對極端熱門的 key，我們可以用機率性提前過期（probabilistic early expiration）或請求合併，讓只有一個請求去查資料庫，其他請求等待那個結果。」

不需要列舉所有可能的問題。挑一兩個對你正在設計的系統最相關的，說明你會怎麼處理。對 Staff 級面試者，把重心放在重要但不那麼顯而易見的場景，而不是在面試官本來就知道你懂的基礎問題上浪費時間。

總結

快取是當資料庫讀取太慢或太昂貴時的解法。它把頻繁存取的資料放在快速的記憶體裡，讓大多數讀取能完全跳過資料庫。

核心取捨很簡單：快取讓讀取更快、降低後端的負載，但引入了資料過時和失效的複雜度。快取的資料可能和資料庫不同步；快取故障如果沒有準備好，可能讓你的資料庫被打垮；熱 key 就算在分散式快取裡也可能製造瓶頸。

在面試中，先確認資料庫瓶頸，再提出快取。說清楚你要快取什麼、使用哪種快取模式、怎麼處理淘汰，以及出問題時怎麼辦。靜態媒體用 CDN 快取，極端熱 key 可以考慮行程內快取。

最重要的是，不要快取所有東西。展示你知道什麼時候快取值得那份複雜度，什麼時候一個 index 設計良好的資料庫就已經夠了。

自我測驗

Q1：將以下快取位置依延遲由低到高排序：External Cache（Redis）、CDN、In-Process Cache。

> **In-Process Cache** < **External Cache (Redis)** < **CDN**
>
> - In-Process Cache：零網路開銷，直接從行程記憶體讀取。
> - External Cache (Redis)：同區域 <1ms，但仍有網路呼叫。
> - CDN：20-40ms，但已比直連 origin server（250-300ms）快很多。

bunkmoat.org

Q2：Cache-Aside 和 Write-Through 的核心差異是什麼？

> • **Cache-Aside**：Application 自己管理快取。讀取時先查快取，miss 時自己去 DB 讀取並寫入快取。快取和 DB 是分開管理的。
> • **Write-Through**：Application 寫入快取，快取**同步**寫入 DB。兩者都完成才算寫入成功。快取資料永遠是最新的，但寫入較慢。

Q3：什麼是 Cache Stampede？最有效的解決方案是什麼？

> **Cache Stampede（快取雪崩）**：熱門的快取項目過期時，大量請求同時 miss 快取，直接打到資料庫，可能把 DB 打垮。
>
> 最有效的解決方案是 **Request Coalescing（請求合併 / Single Flight）**——只讓一個請求去重建快取，其他請求等待那個結果。

Q4：以下關於 eviction policy 的敘述，何者錯誤？(A) LRU 移除最久沒被存取的資料 (B) LFU 移除存取次數最少的資料 (C) FIFO 是生產環境最常用的策略 (D) TTL 通常搭配 LRU 或 LFU 一起使用

> **答案：(C)**
>
> FIFO 很少在生產環境使用，因為它完全忽略使用模式，可能移除正在被頻繁存取的項目。**LRU 才是最常用的預設策略**。

Q5：如果你的系統有一個極端熱門的快取 key（例如名人的個人資料），可能造成什麼問題？如何解決？

> **問題**：Hot Key — 單一 key 收到大量流量，可能讓某個 Redis 節點過載成為瓶頸。
>
> **解決方案**：
>
> 1. **複製熱 key**：把同樣的值存在多個快取節點上，分散讀取負載。
> 2. **加行程內備援快取**：把極端熱門的值存在 application 行程記憶體中，避免一直打 Redis。
> 3. **套用 Rate Limiting**：對異常流量模式踩煞車。

Q6：面試中介紹快取策略的五個步驟是什麼？

> **確認瓶頸**：指出快取要解決的具體問題。**決定快取什麼**：選擇讀取頻繁、不常變動、取得成本高的資料。**選擇快取架構**：選一個符合一致性需求的快取模式（預設 cache-aside）。**設定淘汰策略**：說明如何管理快取大小（預設 LRU + TTL）。**說明缺點**：討論快取失效、快取故障、快取雪崩等取捨。

buildmoat.org